

TD 1 : Algèbre relationnelle, requêtes SQL simples

Algèbre relationnelle et requêtes SQL simples sur le schéma `world`

2026-09-18

 Avec solutions

L3 MIASHS
Université Paris Cité
Année 2025
[Course Homepage](#)
[Moodle](#)



Introduction

Paramétrage du client

En suivant le [guide dBeaver](#) :

- Paramétrez la connexion au serveur dans le client dBeaver.
- Utilisez l'onglet **Projet**.
- Créez un sous-répertoire `tp1` dans le répertoire par défaut du projet.
- Ouvrez un nouveau script dans votre projet et renommez-le en `ex1.sql`.

Schémas

Les schémas sont une abstraction spécifique à PostgreSQL. Malheureusement, cet usage se télescope avec le concept de schéma d'une relation ou d'une table, terme vu en cours.

Dans le jargon PostgreSQL, un "schéma" correspond à la notion habituelle de base de données, c.a.d. un ensemble de tables d'un même système d'information, et un "catalogue" regroupe différents "schémas".

Dans ce TP, nous allons nous concentrer sur le schéma `world` qui contient des informations concernant des villes et des pays/territoires du monde entier. Au cours de ce semestre, nous serons amenés à utiliser d'autres schémas : `pagila` qui contient des informations concernant des films, ...

Dans dBeaver, vous pouvez lister les schémas du catalogue `bd_2023-24`, en développant la connexion `bd_2023-24`.

Puis en développant le schéma `world`, vous pouvez lister les tables qu'il contient.

Vous pouvez consulter la [page de présentation du schéma world](#).

Résolution des noms de tables et d'attributs

Pour résoudre les noms de tables et d'attributs, le serveur SQL utilise la syntaxe `nom_schéma.nom_table[.nom_attribut]` (rappel : les crochets signifient optionnel).

Par exemple, `world.city` désigne la table `city` du schéma `world`.

Pour alléger cette écriture, vous pouvez utiliser le *chemin par défaut*.

Afficher le chemin par défaut avec la requête SQL : `SHOW search_path ;`

Vous pouvez modifier ce chemin avec la requête `SET search_path TO nom_schéma1 [, nom_schéma2]`. Il peut y avoir plusieurs schémas dans le `search_path`.

L'ordre est important. Avec `SET search_path TO world, public ;` la résolution des noms se fera d'abord dans le schéma `world` puis en cas d'échec, dans le schéma `public`.

Après cette instruction `city` désigne la table `city` du schéma `world` si elle existe, sinon cela désigne la table `city` du schéma `public`, sinon le serveur renverra une erreur.

On ajoute systématiquement `public` dans le `search_path`, pour permettre une bonne compatibilité car de nombreuses applications et outils s'attendent à ce que les objets (tables, vues, fonctions) soient accessibles via ce schéma.

Exécutez le script

```
SHOW search_path ;
SET search_path TO world, public ;
SHOW search_path ;
```

Tables

Pour voir le schéma d'une table avec le client dBeaver, il suffit de dérouler le schéma `world` puis de double-cliquer sur la table.

Vous pouvez aussi visualiser toutes les tables du schéma par un clic droit sur le schéma puis **Voir le diagramme**.

Dans la deuxième partie du TP, on s'intéresse aux requêtes, c'est-à-dire, les moyens d'extraire une information pertinente d'une base de données.

Écriture d'une requête

Pour extraire des informations d'une base de données, on utilise l'algèbre relationnelle (pour la théorie) et le langage SQL (pour la pratique).

NB : pensez à documenter vos scripts en utilisant les commentaires SQL : `--` en début de ligne.

Requêtes monotables

On rappelle que la requête SQL

```
SELECT colonne1, colonne2, ...
FROM table
WHERE condition;
```

affiche les colonnes `colonne1`, `colonne2`, ... de `table` pour les lignes qui respectent `condition`.

C'est la transcription de

$\Pi(\sigma(\text{table}, \text{condition}), \text{colonne1}, \text{colonne2})$

Par exemple :

```
SELECT name_country, lifeexpectancy
FROM country
WHERE lifeexpectancy < 50 and continent = 'Asia';
```

affichera le nom et l'espérance de vie des pays d'Asie ayant une espérance de vie inférieure à 50 ans.

Remarquez l'utilisation des apostrophes simples pour délimiter les chaînes de caractères ('Asia').

Exercice 1

Ecrire des requêtes en algèbre relationnelle et en SQL pour extraire les informations suivantes du schéma `world` :

1. Quelles sont les régions ? On ne veut pas de doublons. (25 lignes)

L'algèbre relationnelle vue en cours opérant sur les ensembles, `DISTINCT` est toujours implicitement implémenté par la requête algébrique. Ce n'est pas le cas en SQL (variante multi-ensembliste de l'algèbre relationnelle), il existe donc un opérateur d'élimination des doublons : `DISTINCT`.

Solution

```
 $\Pi(\text{country}, \text{region})$   
SELECT DISTINCT region  
FROM country ;
```

2. Quelles sont les régions situées en Europe (pas de doublons) ? (6 lignes)

Solution

```
 $\Pi(\sigma(\text{country}, \text{continent} = \text{'Europe'}), \text{region})$   
SELECT DISTINCT region  
FROM country  
WHERE continent = 'Europe' ;
```

3. Quels sont les pays situés en Europe du sud ('Southern Europe') ? (15 lignes)

Solution

```
 $\Pi(\sigma(\text{country}, \text{region} = \text{'Southern Europe'}), \text{name\_country})$   
SELECT name\_country  
FROM country  
WHERE region = 'Southern Europe' ;
```

4. A partir de la table `countrylanguage`, quels sont les langues qui sont officielles dans au moins un pays ? (102 lignes)

Solution

```
 $\Pi(\sigma(\text{countrylanguage}, \text{isofficial}), \text{language})$   
SELECT DISTINCT language  
FROM countrylanguage  
WHERE isofficial ;
```

5. Quels sont les codes des pays où le français est langue officielle ? (18 lignes) Même question pour les langues de votre choix ?

Solution

```
 $\Pi(\sigma(\text{countrylanguage}, \text{language} = \text{'French'} \wedge \text{isofficial}), \text{countrycode})$   
SELECT countrycode  
FROM countrylanguage  
WHERE language = 'French' AND isofficial ;
```

6. Quelles sont les villes françaises de plus de 200 000 habitants ? (10 lignes)

 Solution

```

 $\Pi(\sigma(\text{city}, \text{countrycode} = \text{'FRA'} \wedge \text{population} > 200000), \text{name}, \text{population})$ 
SELECT name , population
FROM city
WHERE countrycode = 'FRA' AND population > 200000 ;

```

7. Quels sont les pays où une langue est officielle sans être parlée par au moins la moitié de la population ? (92 lignes)

 Solution

```

 $\Pi(\sigma(\text{countrylanguage}, \text{isofficial} \wedge \text{percentage} < 50), \text{countrycode})$ 
SELECT DISTINCT countrycode
FROM countrylanguage
WHERE isofficial AND percentage < 50 ;

```

8. Pour chaque pays européen, calculer la densité, le GNP par habitant, et l'espérance de vie, ordonner par densité décroissante. (46 lignes)

 Solution

On note ρ l'opération de renommage d'attributs. On peut alors écrire la requête algébrique suivante :

```

R1 =  $\Pi(\sigma(\text{country}, \text{continent} = \text{'Europe'}), \text{name\_country}, \text{population\_country} / \text{surfacearea},$ 
       $\text{gnp} / \text{population\_country}, \text{lifeexpectancy})$ 

```

```

R =  $\rho(R1, \text{population\_country} / \text{surfacearea} \mapsto \text{density}, \text{gnp} / \text{population\_country} \mapsto \text{gnp\_per\_hab})$ 

```

```

SELECT name_country , population_country / surfacearea as density ,
       gnp / population_country as gnp_per_hab , lifeexpectancy
FROM country
WHERE continent = 'Europe'
ORDER BY density DESC;

```

9. Quels sont les pays où l'espérance de vie n'est pas inférieure à 77 ans et le pnb par habitant n'est pas supérieur à (0.010) ? (10 lignes)

 Solution

```

 $\Pi(\sigma(\text{country}, \neg(\text{lifeexpectancy} < 77) \wedge \neg(\text{gnp} / \text{population\_country} > 0.01)), \text{name\_country})$ 
ou, de manière équivalente

```

```

 $\Pi(\sigma(\text{country}, (\text{lifeexpectancy} \geq 77) \wedge (\text{gnp} / \text{population\_country} \leq 0.01)), \text{name\_country})$ 

```

```

SELECT name_country
FROM country
WHERE NOT (lifeexpectancy < 77) AND
       NOT (gnp / population_country > 0.01) ;

```

10. En option : quels sont les pays tels que la condition (espérance de vie supérieure ou égale à 77 ans ou PNB par habitant inférieur à (0.01)) n'est pas vérifiée ? (16 lignes)

 Solution

$\Pi(\sigma(\text{country}, \neg((\text{lifeexpectancy} \geq 77) \vee (\text{gnp/population_country} < 0,01))), \text{name_country})$
ou, de manière équivalente
 $\Pi(\sigma(\text{country}, (\text{lifeexpectancy} < 77) \wedge (\text{gnp/population_country} \geq 0.01)), \text{name_country})$

```
SELECT name_country
FROM country
WHERE NOT (lifeexpectancy >= 77 OR (gnp / population_country < 0.01) );
```

Requêtes multi-tables

On peut aussi combiner plusieurs tables. Par exemple,

```
SELECT language
FROM country as co JOIN countrylanguage as l
    ON co.countrycode = l.countrycode
WHERE co.continent = 'Europe';
```

affichera les langues parlées en Europe. Remarquez l'utilisation des **as** pour donner de nouveaux noms aux tables et l'utilisation de **co.countrycode** pour lever l'ambiguïté sur des noms de colonnes qui seraient éventuellement les mêmes.

Exercice 2

Ecrire des requêtes pour les questions suivantes :

11. Quels sont les noms des capitales sud-américaines ? (14 lignes)

 Solution

En toute rigueur la relation qui résulte d'une opération ne peut pas contenir deux attributs de même nom, d'où la nécessité d'utiliser un renommage. Du point de vue de la complexité algorithmique, il est préférable d'effectuer les sélections avant les jointures.

$R1 = \rho(\text{city}, \text{countrycode} \mapsto \text{city_countrycode})$

$R2 = \sigma(\text{country}, \text{region} = \text{'South America'})$

$R = \Pi(R1 \bowtie_{\text{id}=\text{capital}} R2, \text{name})$

En pratique, on suppose que les renommages **attribut** $\mapsto$ **table.attribut** sont faits implicitement chaque fois que nécessaire.

On peut donc écrire plus simplement :

$\Pi(\text{city} \bowtie_{\text{capital}=\text{id}} \sigma(\text{country}, \text{region} = \text{'South America'}), \text{name})$

ou, moins efficace algorithmiquement :

$\Pi(\sigma(\text{city} \bowtie_{\text{capital}=\text{id}} \text{country}, \text{region} = \text{'South America'}), \text{name})$

En SQL, on écrira

```
SELECT name
FROM country JOIN city ON capital = id
WHERE region = 'South America';
```

ce qui équivaut à la requête suivante qui utilise le produit cartésien et la clause **WHERE** pour exprimer la jointure :

```
SELECT name
FROM country , city
WHERE capital = id AND region = 'South America' ;
```

12. Quels sont les noms des pays où le français est langue officielle ? (18 lignes)

 Solution

```
R1 =  $\sigma(\text{countrylanguage}, \text{isofficial} \wedge \text{language} = \text{'French'})$ 
R =  $\Pi(\text{country} \bowtie_{\text{countrycode}} \text{R1}, \text{name\_country})$ 
SELECT co.name_country
FROM country co JOIN countrylanguage l ON co.countrycode = l.countrycode
WHERE l.isofficial AND l.language = 'French' ;
```

13. Quelles sont les langues officielles d'Asie (pas de doublons) ? (38 lignes)

 Solution

```
R1 =  $\sigma(\text{country}, \text{continent} = \text{'Asia'})$ 
R2 =  $\sigma(\text{countrylanguage}, \text{isofficial})$ 
R =  $\Pi(\text{R1} \bowtie_{\text{countrycode}} \text{R2}, \text{language})$ 
SELECT DISTINCT l.language
FROM country co JOIN countrylanguage l ON co.countrycode = l.countrycode
WHERE l.isofficial AND co.continent = 'Asia';
```

14. Quelles sont les pays où l'espagnol est langue officielle et la forme de gouvernement est 'Federal Republic' ? (3 lignes)

 Solution

```
R1 =  $\sigma(\text{country}, \text{governmentform} = \text{'Federal Republic'})$ 
R2 =  $\sigma(\text{countrylanguage}, \text{isofficial} \wedge \text{language} = \text{'Spanish'})$ 
R =  $\Pi(\text{R1} \bowtie_{\text{countrycode}} \text{R2}, \text{name\_country})$ 
SELECT co.name_country
FROM country co JOIN countrylanguage l ON co.countrycode = l.countrycode
WHERE l.isofficial AND
      l.language = 'Spanish' AND
      co.governmentform = 'Federal Republic' ;
```

15. Quels sont les villes de plus de 1 000 000 habitants situées dans un pays où l'espagnol est langue officielle ? Il faut préciser aussi le nom du pays. (28 lignes)

 Solution

```
R1 =  $\sigma(\text{countrylanguage}, \text{isofficial} \wedge \text{language} = \text{'Spanish'})$ 
R2 =  $\sigma(\text{city}, \text{population} > 1000000)$ 
R =  $\Pi(\text{R1} \bowtie_{\text{countrycode}} \text{R2}, \text{name}, \text{name\_country})$ 
SELECT ci.name, co.name_country
FROM country co
      JOIN countrylanguage l ON co.countrycode = l.countrycode
      JOIN city ci ON co.countrycode = ci.countrycode
WHERE l.isofficial AND l.language = 'Spanish' AND ci.population > 1000000 ;
```

16. Quels sont les pays qui ont au moins deux langues officielles ? (38 lignes)

💡 Solution

$$R1 = \sigma(\text{countrylanguage}, \text{isofficial})$$

$$R2 = \sigma(\text{countrylanguage}, \text{isofficial})$$

$$R3 = \sigma(\text{country} \bowtie_{\text{countrycode}} R1 \bowtie_{\text{countrycode}} R2, R1.\text{language} \neq R2.\text{language})$$

$$R = \Pi(R3, \text{name_country})$$

```
SELECT DISTINCT co.name_country
FROM
  country AS co
  JOIN countrylanguage AS l1 ON co.countrycode = l1.countrycode
  JOIN countrylanguage AS l2 ON co.countrycode = l2.countrycode
WHERE l1.isofficial AND l2.isofficial AND l1.language <> l2.language ;
```

On dispose aussi de la syntaxe JOIN ... USING ($a_1, \dots, a_k$) pour faire des jointures naturelles qui ne considèrent que les attributs communs a_1 à a_k .

```
SELECT DISTINCT name_country
FROM country
  JOIN countrylanguage AS l1 USING (countrycode)
  JOIN countrylanguage AS l2 USING (countrycode)
WHERE l1.isofficial AND l2.isofficial AND l1.language <> l2.language ;
```

17. Quels sont les pays qui comportent au moins deux villes de plus de 1 000 000 habitants ? (32 lignes)

💡 Solution

$$R1 = \sigma(\text{city}, \text{population_city} > 1000000)$$

$$R2 = \sigma(\text{city}, \text{population_city} > 1000000)$$

$$R = \Pi(\sigma(\text{country} \bowtie_{\text{countrycode}} R1 \bowtie_{\text{countrycode}} R2, R1.\text{id} \neq R2.\text{id}), \text{name_country})$$

```
SELECT DISTINCT name_country
FROM country AS co
  JOIN city AS c1 ON co.countrycode = c1.countrycode
  JOIN city AS c2 ON co.countrycode = c2.countrycode AND (NOT c1.id = c2.id)
WHERE c1.population > 1000000 AND c2.population > 1000000;
```